LAPORAN PRAKTIKUM PRAKTIKUM STRUKTUR DATA

Disusun oleh

Nama : Muhammad Yusuf Insy

NIM : 2511532003

Dosen Pengampu : DR. Wahyudi,S.T,M.T.

Asisten Praktikum : jovan

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN 2025

Daftar isi

Daftar isi 2
BAB I PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Tujuan 3
1.3 Manfaat 3
BAB II PEMBAHASAN 3
2.1 Langkah Kerja 4
2.2 Uraian dan Analisis 4
BAB III PENUTUP 7
3.1 Kesimpulan 7
Daftar Pustaka 8

BAB I PENDAHULUAN

1.1 Latar Belakang

Queue merupakan salah satu struktur data dasar yang bekerja dengan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Struktur data ini banyak digunakan dalam kehidupan nyata, seperti antrian di bank, rumah sakit, maupun sistem komputer.

Dalam Java, queue dapat diimplementasikan dengan berbagai cara, seperti menggunakan array (circular queue), menggunakan LinkedList, maupun dikombinasikan dengan struktur data lain seperti stack untuk menyelesaikan permasalahan tertentu.

Pada praktikum ini, dilakukan beberapa implementasi queue dengan pendekatan yang berbeda, serta penerapannya dalam manipulasi data.

1.2 Tujuan

1. Memahami konsep dasar queue (FIFO)
2. Mengimplementasikan queue menggunakan array
3. Menggunakan queue bawaan Java (LinkedList)
4. Menerapkan queue dalam studi kasus
5. Membandingkan berbagai implementasi queue

1.3 Manfaat

1. Meningkatkan pemahaman struktur data
2. Melatih logika pemrograman
3. Memahami perbedaan implementasi queue

BAB II PEMBAHASAN

2.1 Langkah Kerja

1. Membuat queue menggunakan array (circular queue)
2. Membuat driver untuk pengujian queue array
3. Menggunakan queue berbasis LinkedList
4. Membuat program reverse queue menggunakan stack
5. Melakukan iterasi queue menggunakan iterator

2.2 Uraian dan Analisis

a. Queue menggunakan Array (Circular Queue)

Queue array diimplementasikan menggunakan array dengan teknik circular. Variabel front dan rear digunakan untuk menunjuk posisi elemen.

🔍 **Analisis per Baris (queueArray_2511532003)**

- int front_2003, rear_2003, size_2003;
  → Menyimpan posisi depan, belakang, dan jumlah elemen.
- int capacity_2003;
  → Menentukan kapasitas maksimum queue.
- int array_2003[];
  → Array sebagai media penyimpanan data.
- Konstruktor:
  - front_2003 = this.size_2003 = 0;
    → Inisialisasi awal queue kosong.
  - rear_2003 = capacity_2003 -1;
    → Rear diset sebelum indeks pertama agar enqueue pertama benar.
- isFull()
  → Mengecek apakah queue penuh.
- isEmpty()
  → Mengecek apakah queue kosong.
- enqueue_2003()
  - rear = (rear + 1) % capacity
    → Konsep circular (muter ke depan lagi).

    - array[rear] = item
      → Menyimpan data.
    - size++
      → Menambah jumlah elemen.
- dequeue_2003()
    - Ambil data dari front
    - front = (front + 1) % capacity
    - size--
- front_2003() & rear_2003()
  → Mengambil elemen tanpa menghapus.
- display()
  → Menampilkan isi queue dari depan ke belakang.

b. Queue menggunakan LinkedList (Java Collection)

Queue diimplementasikan menggunakan interface Queue dan class LinkedList.

🔍 **Analisis per Baris (queueLinkedList_2511532003)**

- Queue<Integer> q = new LinkedList<>();
  → Membuat queue dinamis.
- q.add(i)
  → Menambahkan elemen ke belakang.
- q.remove()
  → Menghapus elemen depan.
- q.peek()
  → Melihat elemen depan tanpa menghapus.
- q.size()
  → Menghitung jumlah elemen.

c. Reverse Queue menggunakan Stack

🔍 **Analisis per Baris (reversedata_2511532003)**

- Queue<Integer> q
  → Struktur FIFO.

- Stack<Integer> s
  → Struktur LIFO.
- while (!q.isEmpty())
  → Memindahkan semua elemen queue ke stack.
- s.push(q.remove())
  → FIFO → LIFO transform.
- while (!s.isEmpty())
  → Mengembalikan ke queue.
- q.add(s.pop())
  → Urutan jadi terbalik.

d. Iterasi Queue menggunakan Iterator

🔍 **Analisis per Baris (IterasiQueue_2511532003)**

- Iterator<String> iterator = q.iterator();
  → Membuat iterator.
- while(iterator.hasNext())
  → Mengecek elemen berikutnya.
- iterator.next()
  → Mengambil elemen satu per satu.

BAB III PENUTUP

3.1 Kesimpulan

1. Queue adalah struktur data dengan prinsip FIFO
2. Queue dapat diimplementasikan dengan berbagai cara
3. Circular queue meningkatkan efisiensi array
4. LinkedList memberikan fleksibilitas
5. Queue dapat dikombinasikan dengan stack untuk operasi lanjutan

Daftar Pustaka

Oracle Java Documentation

Modul Praktikum Struktur Data